

INTEGRATOR GUIDE

Messaging API

This is the short manual we would hand a developer who needs to send SMS from another system. You talk only to Joyce. Joyce talks to Jasmin, waits for delivery reports, and can optionally ping your own webhook.

1. What you are calling

One HTTPS endpoint accepts every send. Joyce looks at the JSON and decides whether this is one message, the same text to many numbers, or a personalised batch. You do not pick a separate bulk URL.

```
POST https://joyce.oddjobs.tech/api/v1/messaging/send/  
Content-Type: application/json
```

Put the workspace token on every request. Either header works:

```
Authorization: Bearer <messaging_api_token>  
X-Joyce-Token: <messaging_api_token>
```

2. Turn it on in Joyce

Open **Workspace settings**, then:

1. Enable **Joyce messaging API** and save. A token is created for you (an existing token is kept).
2. Copy the bearer token. Use the refresh icon only when you want to rotate it.
3. If this workspace uses **My own Jasmin** and you want true bulk, set the REST API base URL (usually port 8080) so Joyce can call `sendbatch`.
4. Optionally set an **external DLR URL**, HTTP method, retry delay, and max retries. Joyce still records the receipt even if that webhook is down.

3. Send SMS

username must be an enabled Jasmin user in this workspace. Numbers work best in international form without a plus, for example 256700000001.

One number

```
{  
  "username": "u1_myuser",  
  "to": "256700000001",  
  "content": "Hello",  
  "from": "JOYCE",  
  "client_message_id": "ord-100-sms-1",  
  "client_batch_id": "campaign-42",  
  "dlr_level": 3,  
  "priority": 0  
}
```

Same text, many numbers

```
{
  "username": "u1_myuser",
  "to": ["256700000001", "256700000002"],
  "content": "Promo text",
  "client_batch_id": "broadcast-99",
  "client_message_ids": ["msg-a", "msg-b"]
}
```

A different line for each person

```
{
  "username": "u1_myuser",
  "client_batch_id": "orders-2026-07-13",
  "messages": [
    {
      "to": "256700000001",
      "content": "Order #100 is ready",
      "client_message_id": "ord-100"
    },
    {
      "to": "256700000002",
      "content": "Order #101 is ready",
      "client_message_id": "ord-101"
    }
  ],
  "from": "JOYCE",
  "dlr_level": 3
}
```

Your own ids (optional, max 128 characters)

client_message_id	One destination (top-level or inside messages[]). Alias: message_id.
client_message_ids	List in the same order as a multi-value to.
client_batch_id	The whole submit. Alias: broadcast_id.

Jasmin still mints its own message UUID (and a REST batch id when bulk goes through sendbatch). Joyce stores your ids and sends them back on DLR forwards so your app never has to speak Jasmin's identifiers.

How Joyce routes the submit

1	Does not matter	Synchronous classic HTTP /send
2 or more	Yes	Chunked Jasmin REST sendbatch
2 or more	No	Celery queue: classic /send, one by one

4. What you get back

200 means Joyce submitted (or queued via REST) in this request. **202** means bulk_async: accepted, workers will hit Jasmin in the background. mode is single, bulk_rest, or bulk_async. The messages array is capped at 100 rows; look up the rest in Operate with batch_id.

```
{
  "batch_id": "a1b2c3d4e5f6g7h8",
  "client_batch_id": "campaign-42",
  "mode": "single",
  "message_count": 1,
  "jasmin_batch_ids": [],
  "messages": [
    {
      "id": 42,
      "to": "256700000001",
      "status": "submitted",
      "client_message_id": "ord-100-sms-1",
      "client_batch_id": "campaign-42",
      "jasmin_msg_id": "...",
      "error": null
    }
  ],
  "messages_truncated": false
}
```

5. Delivery reports

Jasmin always tells Joyce first. Your webhook is a courtesy copy, not the source of truth.

1. Jasmin calls Joyce at GET | POST `{JOYCE_PUBLIC_BASE_URL}/dlr`.
2. Joyce updates the outbound message row.
3. If you set an external DLR URL, Joyce queues a forward (GET or POST).
4. Failed forwards retry (default 5 times, 60 seconds apart). A dead webhook never blocks internal DLR handling.

What we send your webhook

POST is a JSON body. GET uses the same fields as query parameters. Jasmin's original DLR keys are included, plus:

client_message_id	Your optional id from send
client_batch_id	Your optional broadcast id from send
joyce_message_id	Joyce OutboundMessage primary key
joyce_batch_id	Joyce batch id for the submit
jasmin_msg_id	Gateway UUID (id on the raw DLR)
jasmin_batch_id	REST chunk batch id, when bulk used REST
to / from	Addresses
status / dlr_status	Joyce status and the raw DLR status

In your app, match on `client_message_id`, or on `client_batch_id` plus `to`. You do not need Jasmin to accept your ids.

6. REST bulk progress

When Joyce uses `sendbatch`, it registers `{JOYCE_PUBLIC_BASE_URL}/batch-callback`. Jasmin calls that URL per item so Joyce can store `jasmin_msg_id` before the DLR arrives. You do not call this URL yourself.